Unit 4

Project Closing Phase & Agile Frameworks

4.1 Experiment 1: Project Audits

4.1.1 Project Audits: Objectives and Goals

A project audit is a systematic review of a project's management, processes, and deliverables to assess compliance with standards, identify improvement opportunities, and verify project status [2]. The primary objectives of audits for the Sanskrit NLP Framework project are:

1. Verify compliance with Panini grammar accuracy standards
2. Assess adherence to schedule and budget baselines
3. Identify process improvements for future sprints
4. Validate quality metrics and test coverage
5. Ensure configuration management discipline

4.1.2 Types of Audits

Table 4.1: Types of Audits for Sanskrit NLP Project

Audit Type	**Frequency / Trigger**	**Responsible Party**
Technical audit (code quality, Panini rule accuracy)	End of each sprint	QA Lead + Linguist
Financial audit	Quarterly	Finance Controller

Security audit (API, user data)	Before v1.0 release	External Security Consultant
Process audit (Agile compliance)	Monthly	Scrum Master
Quality assurance audit (test coverage, documentation)	Bi-weekly	QA Lead
Configuration management audit	Per release	Configuration Manager

4.1.3 Audit Checklist (Sample – Technical Audit)

The following checklist is used for technical audits at the end of each sprint:

1. Are Panini sutras correctly mapped to code? (Validation by linguist)
2. Is test coverage $> 85\%$? (Measured by pytest-cov)
3. Are all configuration items version controlled in GitHub?
4. Is code duplication $< 5\%$? (Measured by radon or flake8)
5. Are API responses under 2 seconds for 100-word input?
6. Is there at least one linguist review per sprint?
7. Are all critical defects fixed before release?
8. Is documentation updated for all new features?

4.1.4 Audit Findings (Sprint 3)

Table 4.2: Audit Findings and Corrective Actions

Area	**Issue**	**Severity**	**Corrective Action**
Panini rule mapping	8 sutras incorrectly implemented	High	Re-implement with linguist pair review
Test coverage	Only 72% (target 85%)	Medium	Add 20 more unit tests for edge cases

Documentation	API docs missing authentication section	Low	Complete within 3 days
Code duplication	7% duplication in morphological analyzer	Medium	Refactor shared functions
CI/CD pipeline	Deployment step failing intermittently	High	Fix environment variables

4.1.5 Audit Remediation Tracking

Table 4.3: Audit Remediation Tracking

Issue	Assigned To	Due Date	Status	Verified By
8 sutras incorrect	NLP Engineer 1 + Linguist	Sprint 3 + 2 days	Closed	QA Lead
Test coverage low	NLP Engineer 2	Sprint 4	In Progress	-
API documentation	NLP Engineer 3	Sprint 3 + 3 days	Closed	PM
Code duplication	NLP Engineer 1	Sprint 4	In Progress	-
CI/CD pipeline	DevOps Engineer	Sprint 3 + 1 day	Closed	QA Lead

4.1.6 Project Termination

Project termination occurs after successful completion of all deliverables and acceptance criteria. The termination process for the Sanskrit NLP Framework includes:

1. **Product Verification**: Final accuracy validation by Sanskrit linguists
2. **Documentation Handover**: Complete technical and user documentation
3. **Knowledge Transfer**: Training sessions for client technical team
4. **Financial Closure**: Final billing and budget reconciliation
5. **Lessons Learned**: Documentation of successes and improvement areas
6. **Stakeholder Sign-off**: Formal acceptance from sponsor and users

4.1.7 Software Testing and Testing Strategies

Types of Testing

Table 4.4: Types of Testing for Sanskrit NLP Project

Test Type	**Purpose**	**Tool/Method**
Unit Testing	Validate individual components	pytest
Integration Testing	Test component interactions	pytest + FastAPI
System Testing	End-to-end functionality	Custom test suite
Acceptance Testing	Validate against requirements	Linguist validation
Regression Testing	Ensure no new defects	GitHub Actions
Performance Testing	Verify response times	locust
Security Testing	Identify vulnerabilities	bandit, OWASP ZAP
Usability Testing	Evaluate user experience	User feedback sessions

4.1.8 Evaluation of Project

Project evaluation is conducted at three levels:

Level 1: Technical Evaluation

- Morphological analysis accuracy $\geq 92\%$
- API response time ≤ 2 seconds
- Successful batch processing of Mahabharata corpus

Level 2: Managerial Evaluation

- Schedule variance within $\pm 5\%$
- Cost variance within $\pm 5\%$
- Stakeholder satisfaction $\geq 85\%$

Level 3: Business Evaluation

- Adoption by ≥ 15 academic institutions in Year 2
- Revenue generation of INR 22.5 Lakhs in Year 2
- Positive return on investment by Year 3

4.2 Experiment 2: Agile Approaches Framework – Sprint Planning, Review

4.2.1 Introduction to Agile Process Development

Agile software development is an iterative approach that emphasizes flexibility, collaboration, and customer feedback [14]. Unlike traditional waterfall models, Agile delivers working software in small, frequent increments.

4.2.2 Manifesto of Agile Process

The Agile Manifesto [14] values:

1. **Individuals and interactions** over processes and tools
2. **Working software** over comprehensive documentation
3. **Customer collaboration** over contract negotiation
4. **Responding to change** over following a plan

4.2.3 Agile Principles

The 12 Agile principles applied to the Sanskrit NLP project:

Table 4.5: Agile Principles and Their Application

Principle	Application in Sanskrit NLP Project
1	Highest priority: satisfy customer through early and continuous delivery of Panini rule engine
2	Welcome changing requirements even late in development (new sutras can be added)
3	Deliver working software frequently (weekly API releases)
4	Business people and linguists work together daily
5	Build projects around motivated individuals – trust the team
6	Most efficient communication is face-to-face (daily stand-ups)
7	Working software is the primary measure of progress
8	Agile processes promote sustainable development – no overtime culture

9	Continuous attention to technical excellence and good design
10	Simplicity – maximize work not done
11	Best architectures emerge from self-organizing teams
12	Team reflects on how to become more effective (sprint retrospectives)

4.2.4 Agile Practices and Methodologies

For the Sanskrit NLP project, the following Agile practices are adopted:

- **Scrum**: 2-week sprints with defined roles and ceremonies
- **Kanban**: Visual workflow management using story boards
- **Extreme Programming (XP)**: Pair programming, TDD, continuous integration
- **Lean**: Eliminate waste, amplify learning, decide as late as possible

4.2.5 Agile Framework – Phases of Development

Table 4.6: Agile Development Phases

Phase	**Activities**	**Output**
Envision	Product vision, roadmap, initial backlog	Product vision statement
Speculate	Release planning, sprint planning	Sprint backlog
Explore	Development, testing, daily stand-ups	Working increment
Adapt	Sprint review, retrospective	Improvements
Close	Release, deployment, handover	Shippable product

4.2.6 Relationship Between Conventional and Agile

Table 4.7: Conventional vs Agile Project Management

Aspect	**Conventional (Waterfall)**	**Agile (Scrum/XP)**

Requirements	Fixed at start	Evolving throughout
Delivery	Single final delivery	Frequent incremental delivery
Customer involvement	At milestones only	Continuous throughout
Documentation	Comprehensive	Just-in-time
Team structure	Hierarchical	Self-organizing
Change management	Formal CCB	Welcomed, prioritized in backlog
Testing	End of project	Continuous throughout
Risk management	Upfront planning	Continuous adaptation

4.2.7 IT Service Management Lifecycle

The project aligns with ITIL (Information Technology Infrastructure Library) service management lifecycle:

1. **Service Strategy**: Define value proposition for academic users
2. **Service Design**: Design API, documentation, support model
3. **Service Transition**: Deploy using CI/CD pipeline
4. **Service Operation**: Monitor API performance, user support
5. **Continual Improvement**: Sprint retrospectives, user feedback

4.2.8 Sprint Structure for Sanskrit NLP Project

Table 4.8: Sprint Structure

Parameter	**Value**
Sprint duration	2 weeks (10 working days)
Team size	6 members (1 PM, 3 NLP engineers, 1 linguist, 1 QA)
Velocity (planned)	30 story points per sprint
Velocity (actual average)	26 story points per sprint
Product Owner	Lead Sanskrit Linguist (Dr. Sharma)
Scrum Master	Project Manager
Definition of Ready	Story has acceptance criteria, linguist approval, estimated

Definition of Done	Code reviewed, tests pass, documentation updated, deployed to staging

4.2.9 Sprint Planning (Sprint 4 Example)

Table 4.9: Sprint Planning Example

User Story	Story Points	Assigned To
As a researcher, I can submit a Sanskrit verse and get morphological analysis	8	NLP Engineer 1
As a linguist, I can add custom sandhi rules via configuration file	5	NLP Engineer 2
As a developer, I can run unit tests locally before commit	3	NLP Engineer 3
As a student, I can view parse tree in visual format	8	NLP Engineer 1 + 2
As a QA engineer, I can run automated regression suite	4	QA Lead
As a product owner, I can review rule coverage metrics	2	NLP Engineer 3
Total Planned	**30**	

4.2.10 Sprint Review (Sprint 4 – Actual)

Table 4.10: Sprint Review Results

User Story	Points	Status
Morphological analysis API endpoint	8	Completed
Custom sandhi rules configuration	5	Completed (except documentation)
Local unit test script	3	Completed
Visual parse tree	8	Partially completed (moved to Sprint 5)
Automated regression suite	4	Completed

Rule coverage metrics dashboard	2	Completed
Total Completed	**22**	

Velocity achieved: 22 points (vs planned 30 points)

4.3 Experiment 3: Daily Scrum Planning, Story Boards Creation, Tracking Progress

4.3.1 Daily Scrum (Stand-up) Format

- **Time**: 9:30 AM daily (maximum 15 minutes)
- **Platform**: Microsoft Teams / Zoom
- **Participants**: Entire Scrum team (6 members)
- **Scrum Master facilitates**

Each team member answers three questions:

1. What did I complete yesterday?
2. What will I do today?
3. Are there any blockers?

4.3.2 Sample Daily Scrum (Sprint 4, Day 3)

Table 4.11: Daily Scrum Example

Member	**Update**	**Blockers**
NLP Engineer 1	Completed sandhi rule parser; today: integrate with API	Awaiting linguist review (expected today)
NLP Engineer 2	Fixed 4 failing unit tests; today: add 5 more tests for edge cases	None
NLP Engineer 3	Completed API documentation; today: start metrics dashboard	None

Linguist	Reviewed 10 Panini sutras; today: review sandhi rules for Engineer 1	None
QA Lead	Tested 2 API endpoints; today: automate regression suite	None
Project Manager	Updated risk register; today: stakeholder status report	None

4.3.3 Story Board Creation (Kanban Style)

Table 4.12: Story Board (Kanban)

TO DO	**IN PROGRESS**	**REVIEW**	**DONE**
Visual parse tree (8 pts)	Sandhi rule API (80% complete)	Custom rules config (In review)	Morph analyzer endpoint (8 pts)
Documentation review	Unit test auto (60% complete)	-	Local test script (3 pts)
-	-	-	Regression suite (4 pts)

4.3.4 Tracking Progress – Sprint Burndown Chart

Table 4.13: Sprint Burndown Data (Sprint 4)

Day	**Remaining Points**	**Ideal Remaining**	**Variance**
0 (Planning)	30	30	0
1	28	27	+1
2	25	24	+1
3	22	21	+1
4	20	18	+2
5	18	15	+3
6	14	12	+2
7	10	9	+1

8	8	6	+2
9	4	3	+1
10 (Review)	2	0	+2

Analysis: Team consistently above ideal burndown line due to incomplete visual parse tree story. Recommended action: Split large stories in future sprints.

4.3.5 Cumulative Flow Diagram (CFD) Metrics

Table 4.14: Cumulative Flow Diagram Data

Day	To Do	In Progress	Review	Done
0	30	0	0	0
3	18	8	2	2
6	10	6	4	10
10	2	0	2	26

4.3.6 Sprint Retrospective – Key Action Items (Sprint 4)

1. Add linguist earlier in code review cycle (before implementation, not after)
2. Reduce maximum story size from 8 points to 5 points for better predictability
3. Automate sandhi rule validation using unit tests
4. Schedule daily 15-minute linguist-engineer sync for complex rules
5. Split visual parse tree story into 2 smaller stories (backend + frontend)

4.4 Experiment 4: Agile Approaches – Extreme Programming, Small Releases, Scrum, Lean

4.4.1 Extreme Programming (XP) Practices Adopted

Table 4.15: XP Practices for Sanskrit NLP Project

XP Practice	How Implemented	Benefit Observed

Pair programming	Complex Panini rule implementation with 2 engineers + 1 linguist	40% reduction in rule errors
Test-driven development (TDD)	Write test first for each morphological rule before implementation	Higher confidence in accuracy
Small releases	Weekly release of API improvements to staging	Faster feedback from linguists
Continuous integration	GitHub Actions runs all tests on every commit	Early defect detection (avg 2 hours)
Refactoring	Monthly code cleanup session	Code duplication reduced from 7% to 4%
Collective code ownership	Any engineer can fix any bug; no silos	Faster bug resolution (avg 6 hours)
On-site customer	Sanskrit linguist available full-time during sprints	Immediate clarification of rules
Sustainable pace	40-hour weeks; no overtime except critical fixes	Consistent velocity (22-28 points)

4.4.2 Small Release Strategy

Table 4.16: Small Release Plan

Release	**Content**	**Frequency**
v0.1.0 (Alpha)	Basic morphological analyzer (100 sutras)	Week 4
v0.2.0 (Alpha)	Sandhi splitting for common patterns	Week 6
v0.3.0 (Beta)	Semantic parser for simple verses	Week 8
v0.4.0 (Beta)	Knowledge extraction module	Week 10
v0.5.0 (Beta)	REST API with 3 endpoints	Week 12
v1.0.0 (Production)	Complete framework with documentation	Week 18

4.4.3 Scrum Framework Integration

Table 4.17: Scrum + XP Integration

Scrum Element	XP Practice Integrated	Outcome
Sprint planning	User stories with acceptance tests (TDD)	Clear definition of done
Daily scrum	Pair programming coordination	Better task allocation
Sprint review	Small release demo to stakeholders	Immediate feedback
Sprint retrospective	Refactoring and process improvement planning	Continuous improvement
Product backlog	Continuous prioritization by linguist (on-site customer)	Highest value delivered first

4.4.4 Lean Principles Applied

Table 4.18: Lean Principles in Sanskrit NLP Project

Lean Principle	Application	Measured Impact
Eliminate waste	Auto-generate documentation from code; no duplicate rule specification	Saved 15 person-hours per sprint
Amplify learning	Weekly knowledge transfer sessions between linguists and engineers	Reduced rule misinterpretation by 60%
Decide as late as possible	API design finalized after 3 sprints of experimentation	Avoided 2 major redesigns
Deliver as fast as possible	CI/CD pipeline deploys to staging within 1 hour of commit	Lead time reduced from 5 days to 4 hours
Empower the team	Linguists can directly fix rule mapping via config files	No dependency on engineers for rule changes
Build integrity in	Automated sandhi validation before merge	Defect escape rate reduced to 12%
See the whole	Burndown chart + CFD visible to all stakeholders	Improved transparency and trust

4.4.5 Scrum, XP, and Lean Comparison

Table 4.19: Comparison of Agile Methodologies

Methodology	Primary Focus	Key Practices Used
Scrum	Project management	Sprints, roles, ceremonies, backlog
XP	Engineering practices	TDD, pair programming, CI, refactoring
Lean	Waste elimination	Value stream mapping, pull systems, Kaizen

4.4.6 Benefits of Hybrid Agile Approach

- **Improved Quality**: TDD and pair programming reduced defect density by 40%
- **Faster Delivery**: Small releases enabled weekly feedback cycles
- **Higher Customer Satisfaction**: Linguist involvement increased accuracy validation
- **Better Adaptability**: Changes welcomed even late in development
- **Reduced Waste**: Lean principles eliminated redundant documentation

4.4.7 Conclusion for Unit 4

Project audits conducted at sprint boundaries identified and resolved critical issues including 8 incorrect Panini sutras and CI/CD pipeline failures. The Agile transformation using Scrum with 2-week sprints yielded measurable improvements: velocity increased from 18 to 26 story points over 8 weeks [14]. Daily Scrum meetings ensured effective coordination, while story boards and burndown charts provided visual progress tracking. XP practices, particularly pair programming for complex Panini rules, reduced implementation errors from 15% to 6%. Lean principles eliminated waste and accelerated delivery. The hybrid Agile approach successfully balanced the need for rigorous Panini grammar accuracy with flexible, iterative development [15].